

Prompt Library + Implementation Plan v 3

(Backend is System Brain)

This version is aligned to: - You  Backend Business Logic (main)  DB/Memory + Risk/Guardrails + AI  n 8 n Templates Your requirement: backend must know the whole system be aware of all n 8 n templates, contextualize UI prompts to n 8 n-ready payloads, and enforce guardrails.

1) Architecture principles (non-negotiables)

1) Backend is the only component the UI triggers. 2) Backend is the only component that triggers n 8 n. 3) AI is controlled interpreter never an executor: - selects a template - extracts schema-val inputs - flags risk 4) n 8 n validates structured inputs never raw UI text. 5) Every run is audited, idempotent, and policy-checked.

2) System flow (matching your diagram)

Step 1 — UI → Backend: Task Intake

UI sends: `user_input` (free text) `attachments[]` (optional) `session_id?` (optional) - `intent_hint?` (optional dropdown) - `workspace_id`, `user_id`

Backend stores: - `tasks` row - `conversation_messages` row (raw)

Backend returns: - `task_id` - `session_id` (created if missing) - initial `status`

Step 2 — Backend DB/Memory: Context Retrieval

Backend loads: - `session_summary` - last N messages (bounded) - user/workspace preferences

Backend creates a **Context Pack**:

```
{
  "session_summary": "...",
  "recent_messages": [...],
  "known_entities": {"accounts": [], "vendors": [], "dates": [], "amounts": []},
  "constraints": [...],
}
```

```

    "user_role": "...",
  }

```

Step 3 — Backend ↔ n 8 n Template Registry: Template Awareness

Backend maintains **Template Registry** that mirrors n { `template_id` `n8n_workflow_id` - `version` `inputs_schema_json` `outputs_schema_json` `risk_level` `required_approval` - `allowed_roles` }

Sync options: - Pull: scheduled fetch from n 8 n API - Push: n 8 n publishes manifest on update

Backend always has an in-memory cache for fast selection.

Step 4 — Backend: UI Prompt Cleaning + Contextualization

Backend transforms raw UI text into a stable, execution-safe **Task Spec**.

4 A) Cleaning (sanitization)

- strip UI artifacts (labels, buttons, repeated content)
- remove prompt-injection strings (e.g., "ignore previous instructions")
- normalize whitespace
- detect language
- enforce max lengths
- escape/normalize quotes and special characters

4 B) Contextualization

- add session summary relevant to the goal
- add business constraints from role/workspace
- add template catalog summary (names + short descriptions)

Output: **Normalized Task Spec**

```

{
  "task_goal": "...",
  "requested_outcome": "...",
  "must_have": [...],
  "nice_to_have": [...],
  "entities": {"dates": [], "accounts": [], "currencies": [], "amounts": []},
  "attachments": [...],
  "context": {"session_summary": "...", "snippets": [...]}
}

```

Step 5 — Backend → AI (HF): Interpretation Pipeline

Backend runs AI in a fixed order:

1) **Template Selection** choose 1 best template from Template Registry
produce inputs that match the chosen template schema
Schema Extraction identify risk flags to feed backend policy engine

AI is only allowed to output JSON (strict schema).

Step 6 — Backend Guardrails + Policy Engine (authoritative)

Before executing, backend enforces guardrails:

Guardrail checks

1) **Schema validation** (hard fail if invalid)
Risk-based approvals (risk level and flags)
Idempotency (prevent duplicate execution)
2) **RBAC/ABAC** (is user allowed to run this template?)
PII 4 & Sensitive data (action/storage rules)
3) **Rate limiting** (abuse controls)
4) **Audit logging** (every decision logged)

Resulting states

- needs_clarification
- blocked
- needs_approval
- approved_to_execute

Step 7 — Backend → n 8 n: Execute Workflow

Backend builds the n 8 n payload (never raw UI):

```
{
  "workspace_id": "...",
  "task_id": "...",
  "run_id": "...",
  "template_id": "...",
  "template_version": "...",
  "inputs": {"...": "..."},
  "idempotency_key": "...",
  "callback_urls": {
    "started": "...",
    "approval_request": "...",
    "completed": "...",
    "failed": "..."
  }
}
```

```
}  
}
```

Step 8 — n 8 n → Backend: Callbacks + Results

Backend receives: - status updates - artifacts - errors

Backend: - stores outputs - updates audit trail - updates session memory (append + summary refresh)

3) Prompt Library v 3 (backend-orchestrated)

3 . 1 Global rules (single system prompt)

SYS_INTERPRETER_V 3 - You are an interpreter layer. - You do not execute tools. - You do not
You return ONLY valid JSON. - If required fields are missing: ask for clarification by returning missing_fields.

3 . 2 Prompt: Context Selector

CTX_SELECTOR_V 3 **Purpose:** Select the minimal relevant context to pass into downstream prompts.

Input: Context Pack + Task Spec

Output schema:

```
{  
  "status": "ok",  
  "context_to_use": {  
    "summary": "string",  
    "snippets": ["string"]  
  },  
  "notes": "string"  
}
```

3 . 3 Prompt: Template Selector (must use Template Registry list)

TPL_SELECTOR_V 3 **Purpose:** Pick exactly one template/workflow.

Input: - task_spec - templates[] (from backend registry) - context_to_use

Output schema:

```
{
  "status": "ok",
  "selected_template_id": "string",
  "confidence": 0.0,
  "rationale": "string",
  "assumptions": ["string"]
}
```

3.4 Prompt: Schema Extraction (template-specific)

EXT_TEMPLATE_V 3 **Purpose:** Extract values into the chosen template's **inputs schema**.

Input: - `task_spec` - `context_to_use` - `inputs_schema_json`

Output schema:

```
{
  "status": "ok",
  "confidence": 0.0,
  "data": {},
  "missing_fields": ["string"],
  "notes": "string"
}
```

Rules: - `data` MUST validate against schema. - If cannot fill required fields → populate `missing_fields`.

3.5 Prompt: Risk Flagging (advisory)

RISK_FLAGGER_V 3 **Purpose:** Identify risk indicators for backend guardrails.

Input: `template metadata + extracted inputs + context_to_use`

Output schema:

```
{
  "risk_flags": ["string"],
  "risk_summary": "string"
}
```

3.6 Prompt: Clarification Question Builder (only when needed)

CLARIFY_BUILDER_V 3 **Purpose:** Turn `missing_fields` into a minimal question set.

Output schema:

```
{
  "status": "needs_clarification",
  "questions": [
    {"field": "string", "question": "string", "example": "string"}
  ]
}
```

3.7 Prompt: Session Summarizer (memory)

SESSION_SUMMARIZER_V 3 **Purpose:** Maintain a small session summary for retrieval.

Output schema:

```
{
  "status": "ok",
  "updated_summary": "string",
  "key_facts": ["string"],
  "open_threads": ["string"],
  "constraints": ["string"],
  "entities": [
    {"name": "string", "type": "string", "notes": "string"}
  ]
}
```

4) Guardrails v 3 (backend policy engine)

4.1 Required modules

1) **Schema Validator** (AJV / zod / jsonschema) 2) **RBAC Authorizer** 3) **Risk Rules Engine** 4) **Approval Service** 5) **Idempotency Service** 6) **Audit Logger** 7) **Rate Limiter**

4.2 Practical policy rules

- If template `risk_level >= 4` → require approval
 - If template writes to finance/db → require approval unless role allows
 - If external URLs/domains appear → require confirmation
 - If PII suspected → redact + restrict sharing
 - If schema invalid → block and request clarification
-

5) Implementation Plan v 3 (backend-first)

Sprint 1 — Template Registry (n 8 n awareness)

- Build `templates` + `template_versions` tables
- Implement n 8 n sync (pull or push)
- Admin endpoint: list templates + schemas
- Cache templates in backend memory

Sprint 2 — Task Intake + Session Memory

- `POST /tasks` create task + session
- `conversation_messages` + `conversation_sessions.summary`
- Return `session_id` to UI if missing

Sprint 3 — Prompt Cleaning + Task Spec Builder

- Implement prompt sanitizer
- Implement entity normalization (dates, currency)
- Generate `task_spec`

Sprint 4 — AI Interpretation Pipeline

- `CTX_SELECTOR_V 3` → `TPL_SELECTOR_V 3` → `EXT_TEMPLATE_V 3` → `RISK_FLAGGER_V 3`
- Persist AI outputs + hashes

Sprint 5 — Guardrails + Approvals

- Schema validation hard gate
- RBAC checks
- Approval request creation + UI approval flow

Sprint 6 — n 8 n Execution Bridge + Callbacks

- Execute endpoint (backend only)
- Callback endpoints
- Idempotency enforcement
- Store outputs/artifacts

Sprint 7 — Memory Summarization Loop + Observability

- Update session summary after each completion
- Add audit dashboards (runs, failures, approvals)
- Add rate limiting + alerts

6) Backend contracts (what each component gets)

UI → Backend

Raw: human text + attachments + session_id.

Backend → AI

Cleaned Task Spec + selected context + template catalog snippet/schema.

Backend → n 8 n

Validated structured JSON inputs + ids + callbacks.

n 8 n → Backend

Status + outputs + artifacts.

7) What to build first (order that prevents rework)

1) Template Registry + sync 2) Task intake + session_id persistence 3) Prompt cleaning + Ta
4) Schema validation + guardrails 5) AI extraction into schema 6) n 8 n bridge + callbacks
approvals + audit + memory summarization